Quantum Linear System Algorithms for Graph-Based Financial Risk Diffusion

Group 26A

Harris Ahmed M, Neha Girdhar, Snigdha Chandan,

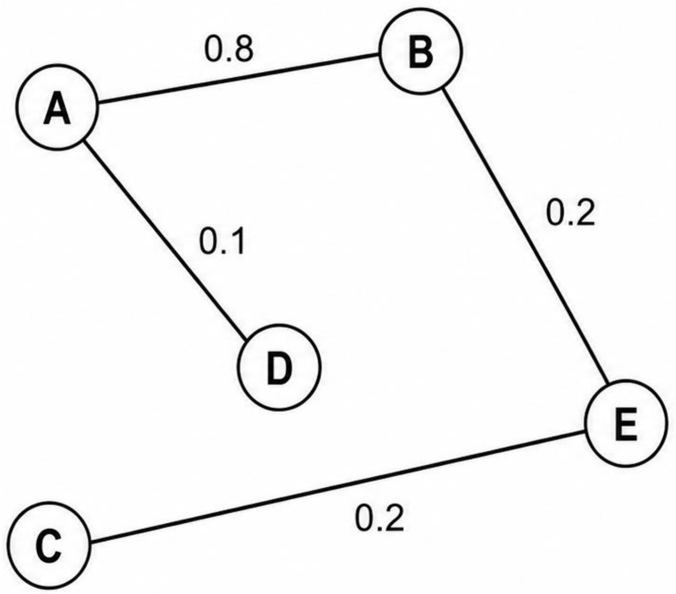
Selvakumar Arumugam, Rishanthi Ramakrishnan

Problem Statement

- Given:
  - A large but sparse financial network with N institutions (banks)
  - A few banks suddenly get high risk scores
- How does the risk propagate across the network?
- Which banks are affected?
  - What are the most affected banks?
  - Is my bank safe?

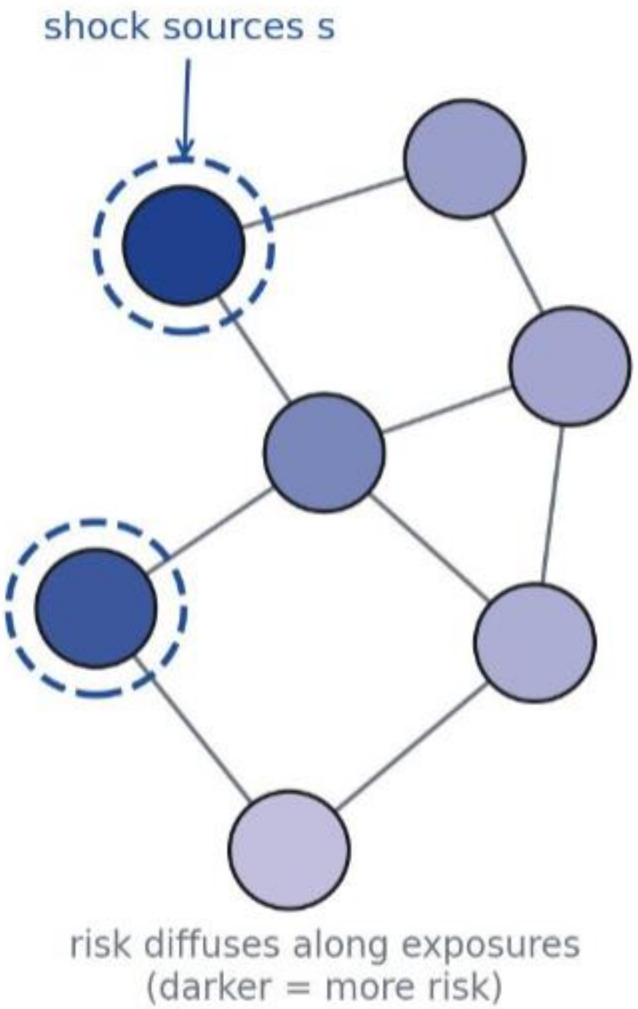
Problem Statement: Example

Financial Network



Adjacency Matrix

$$
J = \begin{array}{c} \\ A \\ B \\ C \\ D \\ E \end{array}
\begin{array}{c}
\begin{array}{ccccc} A & B & C & D & E \end{array} \\
\begin{bmatrix}
0 & 0.8 & 0 & 0.1 & 0 \\
0.8 & 0 & 0 & 0 & 0.2 \\
0 & 0 & 0 & 0 & 0.2 \\
0.1 & 0 & 0 & 0 & 0 \\
0 & 0.2 & 0.2 & 0 & 0
\end{bmatrix}
\end{array}
$$



Problem Statement: Model

- N nodes
- Diffusion Parameter, α
- Adjacency matrix, J
- Degree matrix, D – diagonal : $D_{ii} = \sum_j J_{ij}$
- Laplacian, L = D – J
- Risk scores, x

- Model: $$\min_x \quad \left(\frac{\alpha}{2}\right) x^T L x + \left(\frac{1-\alpha}{2}\right) \|x - c\|^2$$

Problem Statement – 3 Models

- Model 1

$$[\alpha L + (1 - \alpha) I]\, x = (1{-}\alpha) c$$

  - Biased towards nodes with high degree. So, normalize Laplacian.

- Model 3

$$\left[\alpha D^{-1} L + (1 - \alpha) I\right] x = (1{-}\alpha) c$$

  - Matrix not Hermitian, also poorly conditioned.
    So, symmetrically normalize Laplacian.

- Model 2

$$[\alpha L_{\text{sym}} + (1 - \alpha) I]\, x = (1{-}\alpha) c$$

  - Now, the matrix is nice!

$$L_{sym} = D^{-1/2} L D^{-1/2}$$

Quantum Formulation

- Linear equation

$$A\left|x\right\rangle = \left|b\right\rangle$$

- The quantum solution state is proportional to the true solution
- Realistic assumption on input shock vector

$$\left|b\right\rangle = \sum_{j=0}^{\gamma-1} \frac{1}{\sqrt{\gamma}} \left|i_j\right\rangle$$

- Sparsity, s
- Condition Number, κ

Algorithmic Pipeline (Modular Components)

Financial Model

- Adjacency matrix
- Normalized Laplacian
- Linear system

Prepare |b>

(Sparse State Preparation)

2 methods

- Gleinig-Hoefler
- Shukla-Vedula variant

Represent A

- Hamiltonian Simulation (Trotterization)
- Block Encoding (Berry-Childs, LCU)

Invert Eigenvalues

- Polynomial Approximation
- QPE-Based (Traditional HHL, Berry-Childs HHL – 2 variants)

  QSVT-Based

Readout

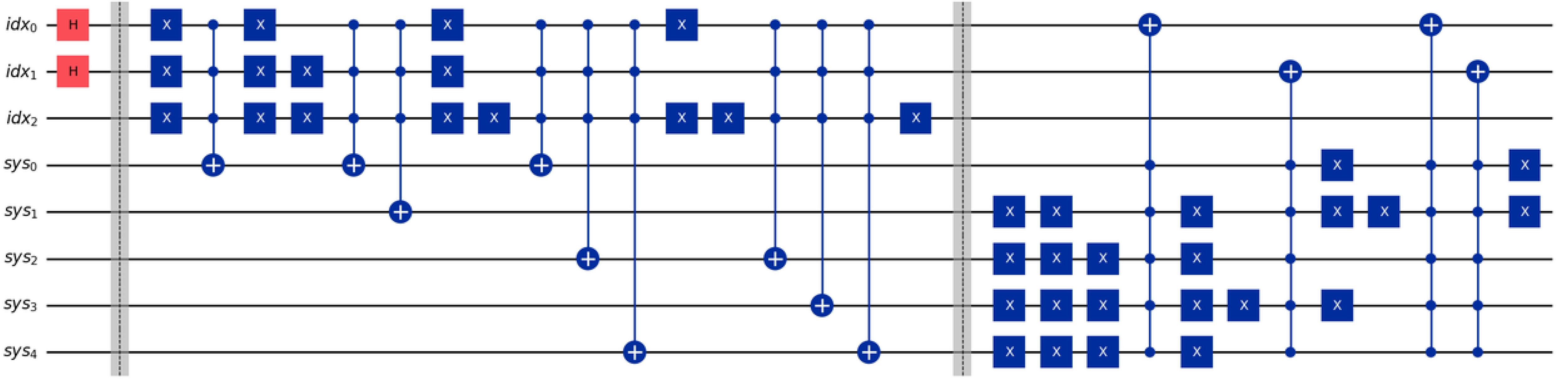
(Measurement)

2 use cases

- Basis sampling (top k)
- Observable estimation (single node risk)

Shock State Preparation Circuit Example

$$|b\rangle = \frac{1}{2}\left(|1\rangle + |3\rangle + |21\rangle + |28\rangle\right)$$



Block Encoding (Low and Chuang, 2016)

- Encode A in a higher dim'l space

- Eigenvalue/Eigenvector pairs

  become

  2D-invariant subspaces

- Each Eigenpair = A Qubit (Qubitization)

Block Encoding

$$U_A = \left(\begin{array}{c:c} \boxed{\dfrac{A}{\alpha_A}} & * \\ \hdashline * & * \end{array}\right)$$

$$(\langle 0| \otimes I\,)\, U_A\, (I \otimes |0\rangle) = \frac{A}{\alpha_A}$$

$$\lambda \to \cos\theta \to \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Block Encoding

- Two kinds
  - Berry-Childs Block Encoding

    R = Reflection about Im(T)
    S = Swap row and system
    W = SR

$$T : |0\rangle_{\text{prepare}} \, |0\rangle_{\text{row}} \, |j\rangle_{\text{system}}$$

$$\longmapsto$$

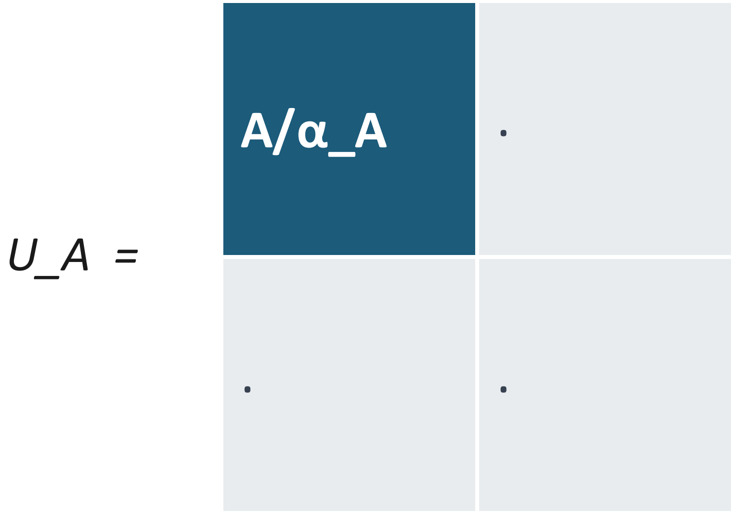
$$\sum_{\ell} \sqrt{\frac{|A_{r_j(\ell),j}|}{\alpha_A}} \, e^{i\phi_{r_j(\ell)j}} \, |\ell\rangle_{\text{prepare}} |r_j(\ell)\rangle_{\text{row}} \, |j\rangle_{\text{system}}$$

$$+\sqrt{1 - \frac{\sum_{\ell} |A_{r_j(\ell),j}|}{\alpha_A}} \, |\text{fail}\rangle_{\text{prepare}} |r_j(\text{fail})\rangle_{\text{row}} |j\rangle_{\text{system}}$$

  - Linear Combination of Unitaries

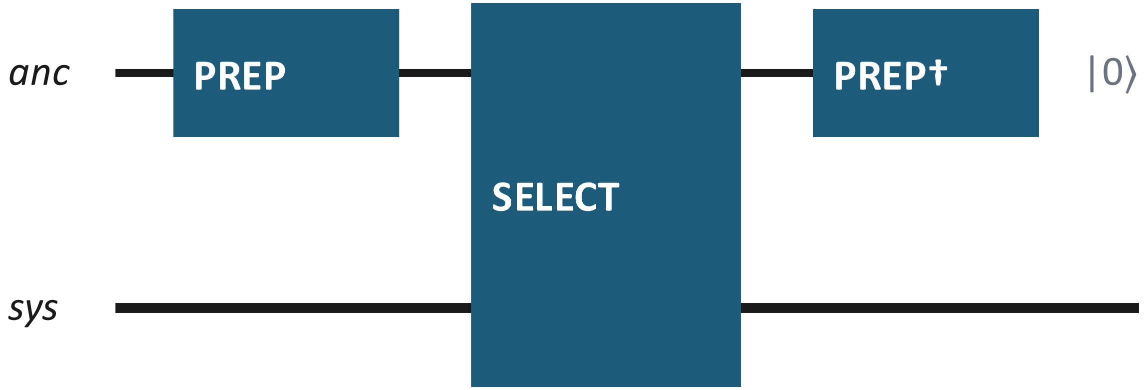
$$A = \sum_i a_i U_i \qquad \text{More generally,} \qquad M = \sum_i a_i U_i$$

LCU Block Encoding



$$(\langle 0|\otimes I)\cdot U\_A\cdot(|0\rangle\otimes I) = A/\alpha\_A$$

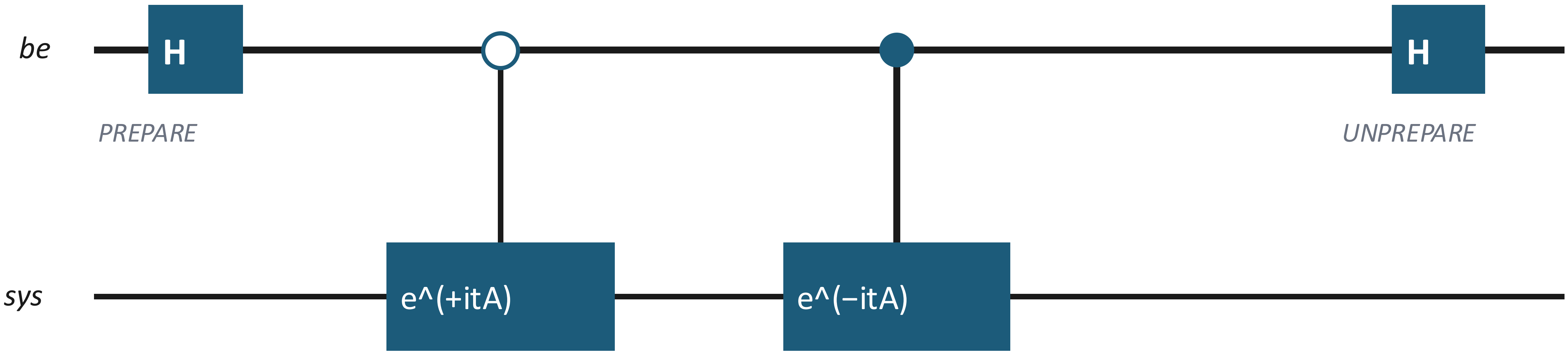
LCU sandwich



$$\langle 0|_a(\mathrm{PREP}^\dagger\cdot\mathrm{SELECT}\cdot\mathrm{PREP})|0\rangle_a = \Sigma_j(\alpha_j/\alpha\_A)U_j = A/\alpha\_A$$

The Implemented LCU Block Encoding

PREPARE = H → SELECT = controlled ±e^{itA} → UNPREPARE = H; ancilla-|0⟩ block = cos(tA)



SELECT (controlled ± evolution)

○ control on |0⟩ → e^(+itA)   ● control on |1⟩ → e^(−itA)

Eigenvalue Inversion Architecture (QPE)

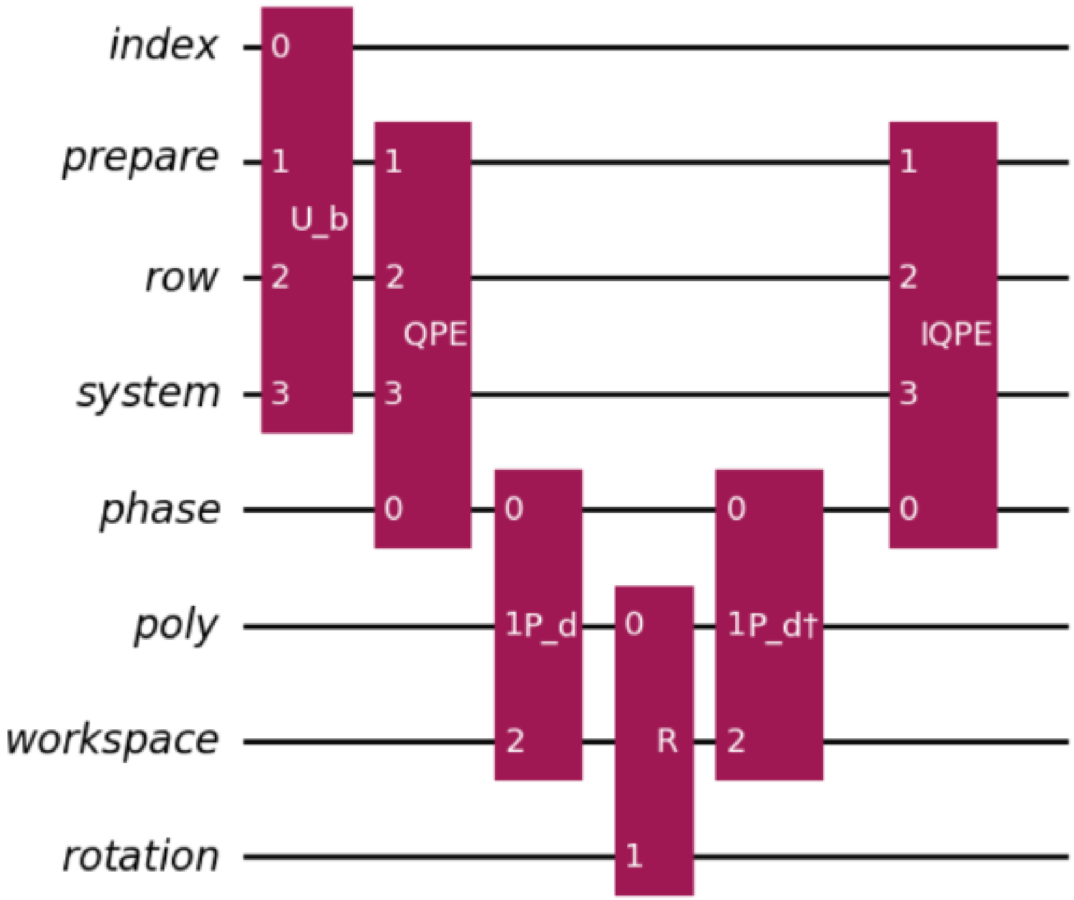


Figure 6: Eigenvalue Inversion Architecture - Variant 1 (Compute, then rotate)

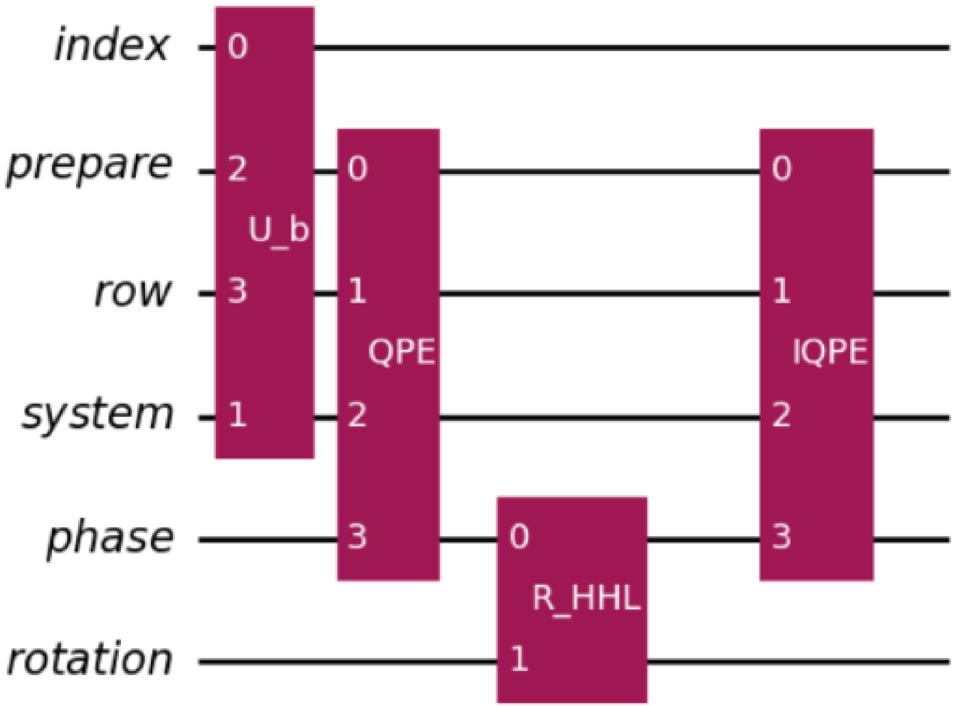
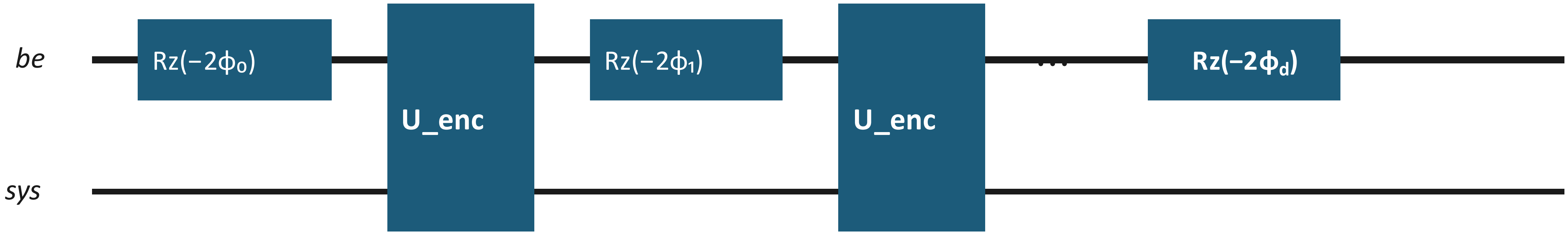


Figure 7: Eigenvalue Inversion Architecture - Variant 2 (Rotate while computing)

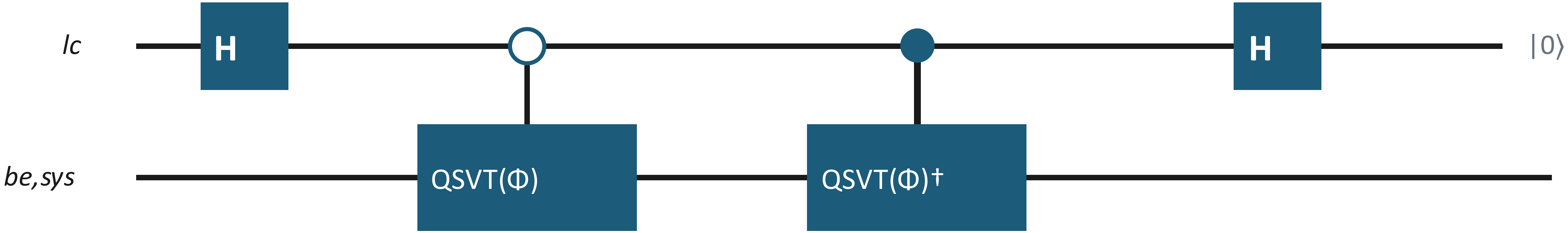
QSVT Consumption & the Real-Part LCU

Phased iterates Rz(−2φ_k) alternate with U_enc; an outer LCU keeps the real part p(cos(tA))

QSVT layer structure

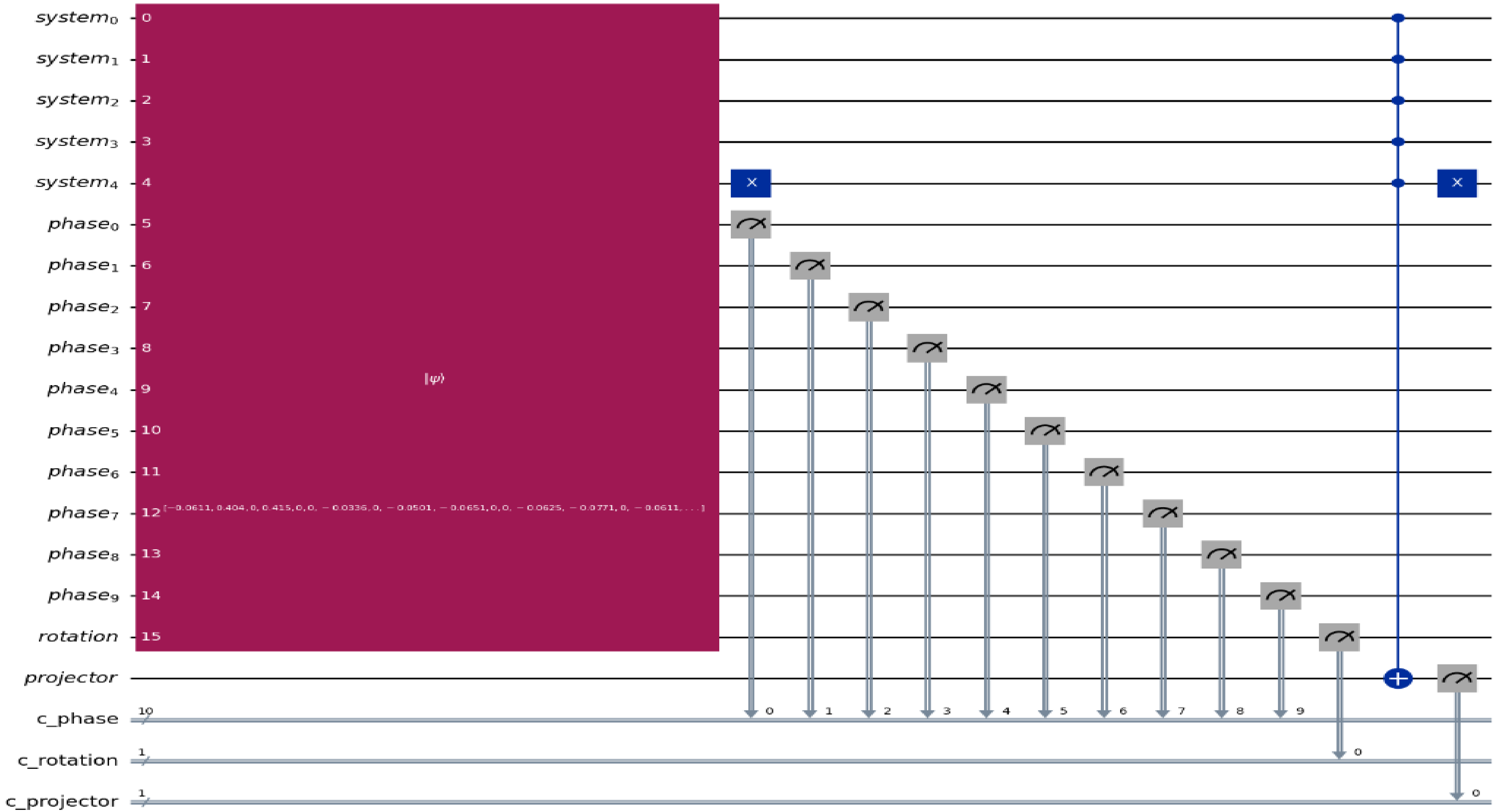


Real-part LCU wrapper



Measurement

```
Target node             : 15
Binary representation   : 01111
```



Results – Comparison and Discussion

Table 1: Comparison of the three quantum linear-system implementations (Model 2, $\alpha = 0.6$).

Metric	QPE Variant 1	QPE Variant 2	QSVT
Fidelity	0.998718	0.998465	0.999966
Post Selection Prob	0.263	0.261	0.381
Block Enc Oracle Calls	$2(2^m - 1)$	$2(2^m - 1)$	d_{qsvt}
Requires Poly Oracle	Yes	No	No
Circuit Depth Estimate	$O(2^m D_{BC} + d_{qpe})$	$O(2^m D_{BC} + 2^m)$	$O(d_{qsvt} D_{LCU})$
Registers used	Index, System, Row, Prepare, Phase, Poly, Rotation	Index, System, Row, Prepare, Phase, Rotation	Index, System, Row, Prepare
Total qubits	34	27	10
Requires QPE	Yes	Yes	No
Controlled rotations	Simple controlled rotations + Block Encoding	2^m multicontrolled rotations + Block Encoding	Block Encoding only
Postselection	Ancilla + Phase register	Ancilla + Phase register	Block-encoding ancillas
Asymptotic complexity	$O\left(\frac{\kappa s}{\epsilon} \text{polylog}(N)\right)$	$O\left(\frac{\kappa s}{\epsilon} \text{polylog}(N)\right)$	$O(\kappa s \log(1/\epsilon) \text{polylog}(N))$

Complexity Comparison with Classical Methods

Table 2: Representative complexity comparison of classical and quantum approaches for graph-based financial risk diffusion.

Method	**Representative Complexity**
Preconditioned Conjugate Gradient (PCG)	$O(E\sqrt{\kappa}\log(1/\epsilon))$
Nearly-linear Laplacian Solver	$O(E\,\mathrm{polylog}(N)\,\log(1/\epsilon))$
Graph Diffusion (e.g., Personalized PageRank)	$O(E\log(1/\epsilon))$ (iterative propagation)
Graph Neural Networks (Inference)	Approx $O(EL)$ for L message-passing layers
QPE-based HHL (implemented)	$O\left(\frac{\kappa s}{\epsilon}\,\mathrm{polylog}(N)\right)$
QSVT-based QLSA	$O(\kappa s\log(1/\epsilon)\,\mathrm{polylog}(N))$

References (key)

[1] Harrow, Hassidim, Lloyd — Quantum algorithm for linear systems of equations, PRL 103, 150502 (2009).

[2] Childs, Wiebe — Hamiltonian simulation using linear combinations of unitary operations, QIC 12, 901 (2012).

[3] Low, Chuang — Hamiltonian simulation by qubitization, Quantum 3, 163 (2019).

[4] Gilyén, Su, Low, Wiebe — Quantum singular value transformation and beyond, STOC 2019.

[5] Xiaojin Zhu, Zoubin Ghahramani, and John D. Lafferty, Semi-supervised learning using gaussian fields and harmonic functions, (20th ICML), 2003, pp. 912–919.

[6] Berry, Childs, Cleve, Kothari, Somma — Truncated Taylor series simulation, PRL 114, 090502 (2015).

[7] Gleinig, Hoefler — An efficient algorithm for sparse quantum state preparation, DAC 2021.

[8] Berry, Childs — Black-box Hamiltonian simulation and unitary implementation, QIC 12, 29 (2012).

[9] Chakraborty, Gilyén, Jeffery — The power of block-encoded matrix powers, ICALP 2019.

[10] Camps, Van Beeumen — FABLE: fast approximate block-encodings, IEEE QCE 2022.

[11] Spielman, Teng — Nearly-linear time Laplacian solvers, STOC 2004.

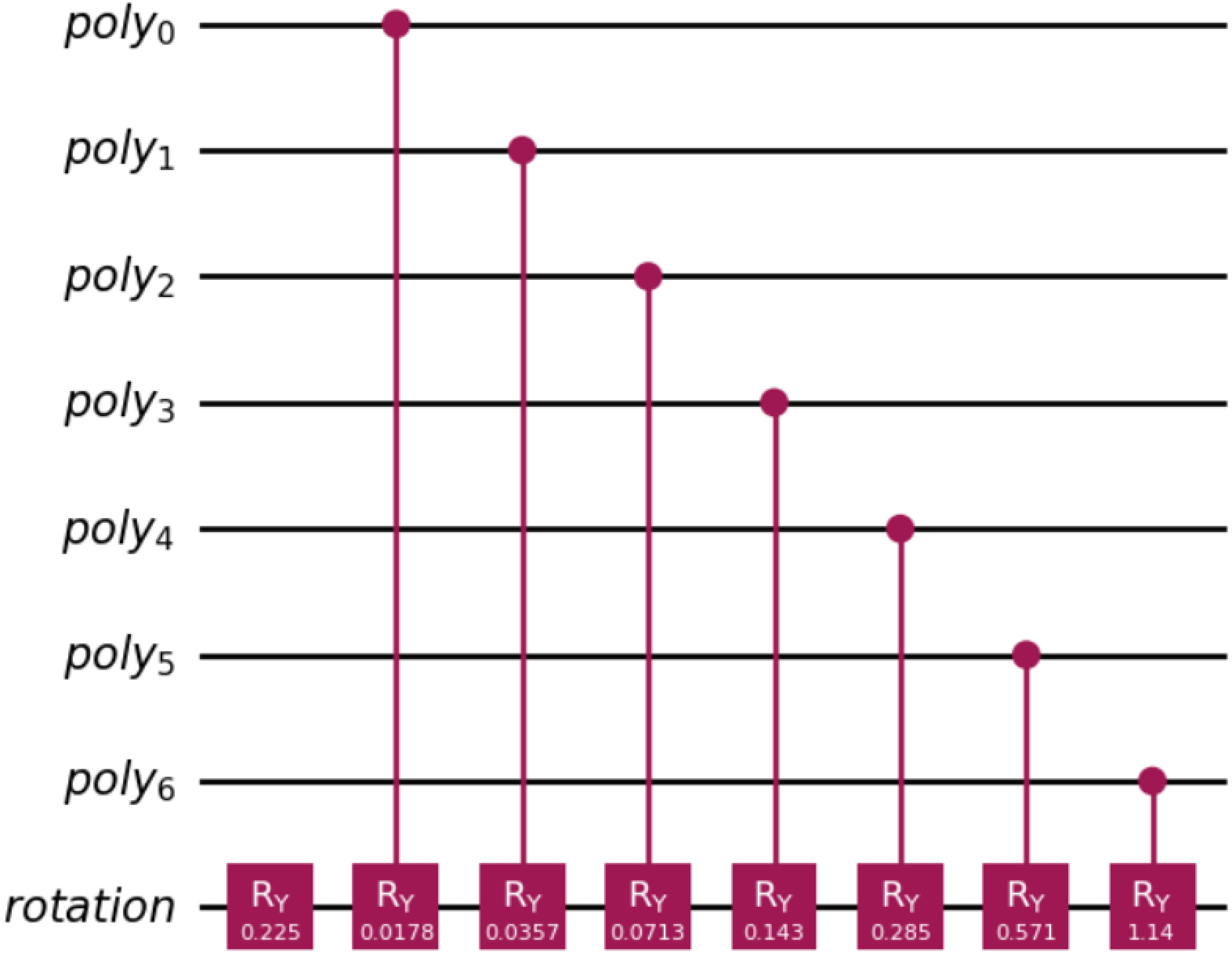
[12] Sachdeva, Zhao — A simple and efficient parallel Laplacian solver, SPAA 2023.

[13] Alok Shukla and Prakash Vedula, An efficient quantum algorithm for preparation of uniform quantum superposition states, arXiv preprint arXiv:2306.11747 (2024).

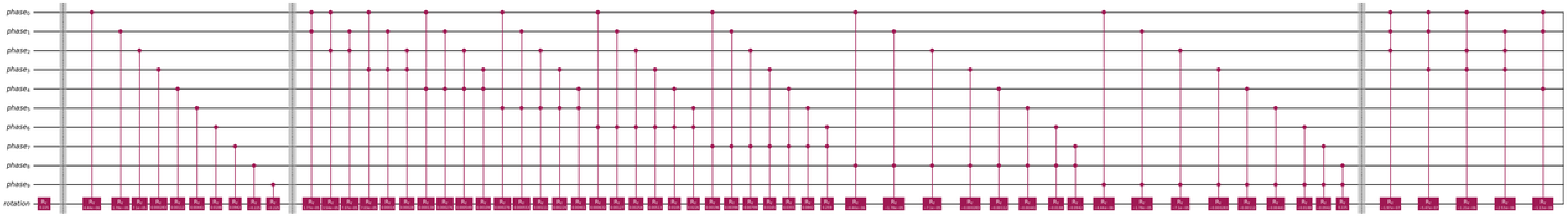
[14] Dengyong Zhou, Olivier Bousquet, Thomas N. Lal, Jason Weston, and Bernhard Schölkopf, Learning with local and global consistency, Advances in Neural Information Processing Systems, vol. 16, 2004.

Questions?

Thank you!



poly0
poly1
poly2
poly3
poly4
poly5
poly6
rotation
RY 0.225
RY 0.0178
RY 0.0357
RY 0.0713
RY 0.143
RY 0.285
RY 0.571
RY 1.14


$phase_0$
$phase_1$
$phase_2$
$phase_3$
$phase_4$
$phase_5$
$phase_6$
$phase_7$
$phase_8$
$phase_9$
$rotation$